

BOOTCAMP PROJECT 2

DATABRICKS

Notebook 1: ADLS gen2 Connectivity using Service Principle Method

Steps:

- First go the entra id -> App registration, then create a user and then click create secret and copy the secret value. This is the SecretID
- Then copy the AppId , TenantID and the name of the serviceprinciple
- Then go to Key Vault and create secrets for all these ID's.
- Then go to the IAM of Key Vault add add the KeyVault Secret User / KeyVault Administrator for the member AzureDatabricks.
- Then go to the storage account -> IAM and select storage blob data contributor and select the service principle name in the members option.
- Then open the Databricks workspace and edit the url.
- Keep everything before .net/ and add #secrets/createScope.
- Enter the name for the scope and copy paste the Vault URI and ResourceID.
- Then open a notebook and add the following code:

```
dbutils.secrets.list(scope="KeyVaultConnection")
appid = dbutils.secrets.get(scope="KeyVaultConnection", key="appid")
secretid = dbutils.secrets.get(scope="KeyVaultConnection", key="secretid")
tenantid = dbutils.secrets.get(scope="KeyVaultConnection", key="tenantid")
storage_account="tarunadlsproject1"

spark.conf.set(
    f"fs.azure.account.auth.type.{storage_account}.dfs.core.windows.net",
    "OAuth"
)

spark.conf.set(
    f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net",
    appid
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.core.windows.net",
    secretid
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.{storage_account}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenantid}/oauth2/token"
)
```

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

🔍 vault administrator

Type: All

Category: All

Name ↑	Description ↑	Type ↑
Key Vault Administrator	Perform all data plane operations on a key vault and all objects in it, including certificates, keys, and secrets. Cannot manage key vault resources or manage role assignments. Only works for key vaults that use the ...	BuiltInRole
Key Vault Data Access Administrator	Manage access to Azure Key Vault by adding or removing role assignments for the Key Vault Administrator, Key Vault Certificates Officer, Key Vault Crypto Officer, Key Vault Crypto Service Encryption User, Key Vault...	BuiltInRole
Oracle.Database Exascale Storage Vault Administrator	Grants full access to manage Exascale Storage Vaults	BuiltInRole

Showing 1 - 3 of 3 results.

Home > Key vaults > tarunadlsproject1 > Access control (IAM)

Add role assignment

Role **Members** Conditions Review + assign

Selected role Key Vault Administrator

Assign access to ☒ User, group, or service principal

☐ Managed identity

Members [+ Select members](#)

Name	Object ID	Type
No members selected		

Select members

🔍 azuread

☒ AzureDataBricks Application

```
dbutils.secrets.list(scope="KeyVaultConnection")

[SecretMetadata(key='appid'),
 SecretMetadata(key='azuresqldbpwd'),
 SecretMetadata(key='secretid'),
 SecretMetadata(key='tenantid')]
```

```
appid = dbutils.secrets.get(scope="KeyVaultConnection", key="appid")
secretid = dbutils.secrets.get(scope="KeyVaultConnection", key="secretid")
tenantid = dbutils.secrets.get(scope="KeyVaultConnection", key="tenantid")
```

```
spark.conf.set("fs.azure.account.auth.type.tarunadlsproject1.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.tarunadlsproject1.dfs.core.windows.net", "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.tarunadlsproject1.dfs.core.windows.net", "appid")
spark.conf.set("fs.azure.account.oauth2.client.secret.tarunadlsproject1.dfs.core.windows.net", "secretid")
spark.conf.set("fs.azure.account.oauth2.client.endpoint.tarunadlsproject1.dfs.core.windows.net", "https://login.microsoftonline.com/tenantid/oauth2/token")
```

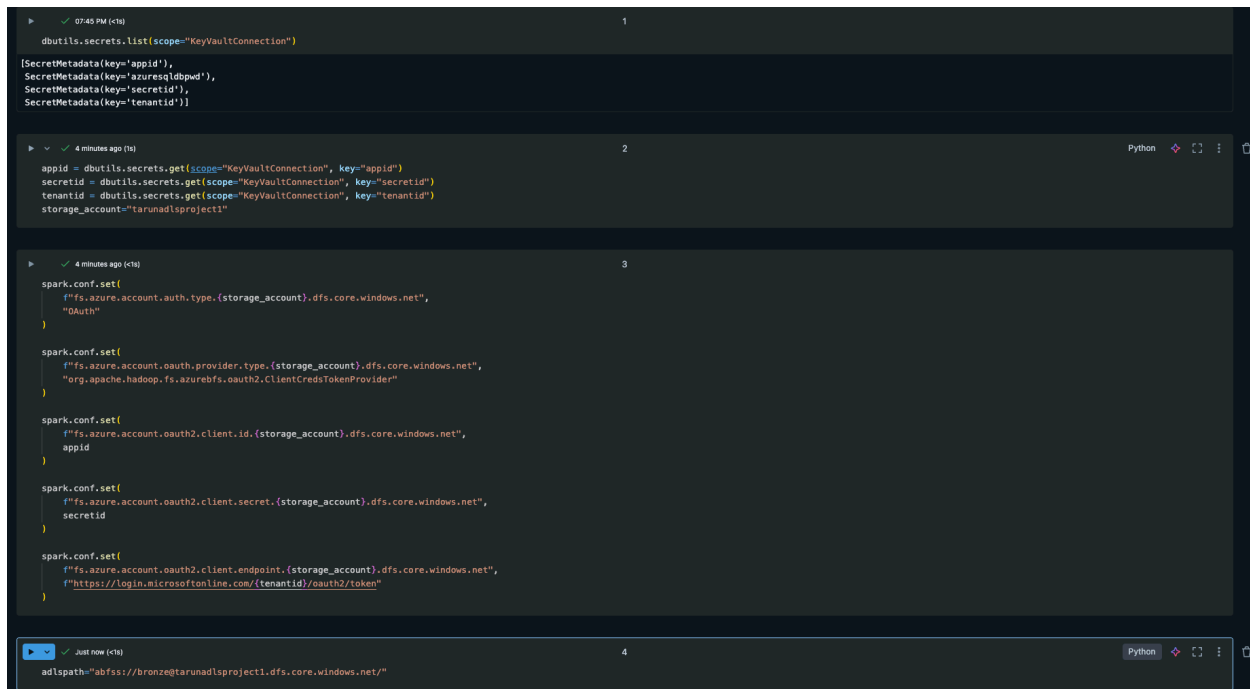
```
adlspath="abfss://bronze@tarunadlsproject1.dfs.core.windows.net/"
```

```
1 files = dbutils.fs.ls(adlspath)
2
3 for f in files:
4     print(f.name)
```

ExecutionError: (shaded.databricks.azurebfs.org.apache.hadoop.fs.azurebfs.exceptions.AbfsRestOperationException) HTTP Error 400: url='https://login.microsoftonline.com/tenantid/oauth2/token' AADToken: HTTP connection to https://login.microsoftonline.com/tenantid/oauth2/token failed for getting token from AzureAD: requestid='23d5b333-f894-492c-b31b-5cda68f0100'; contentType='application/json; charset=utf-8'; response '{"error":{"error_description":"AADSTS900023: Specified tenant identifier 'tenantid' is neither a valid DNS name, nor a valid external domain. Trace ID: 23d5b333-f894-492c-b31b-5cda68f0100 Correlation ID: b65e9027-2ec5-469c-bab5-a29f52cb9c4c Timestamp: 2026-02-12 00:53:15Z","error_codes":[900023],"timestamp":"2026-02-12 00:53:15Z","trace_id":"23d5b333-f894-492c-b31b-5cda68f0100","correlation_id":"b65e9027-2ec5-469c-bab5-a29f52cb9c4c","error_uri":"ht...

[Diagnose error](#) [Debug](#)

[Assistant Quick Fix ON](#)



```
07:45 PM (x16) 1
dbutils.secrets.list(scope="KeyVaultConnection")

[SecretMetadata(key='appid'),
 SecretMetadata(key='azuresqlbpwd'),
 SecretMetadata(key='secretid'),
 SecretMetadata(key='tenantid')]

4 minutes ago (76) 2
Python
appid = dbutils.secrets.get(scope="KeyVaultConnection", key="appid")
secretid = dbutils.secrets.get(scope="KeyVaultConnection", key="secretid")
tenantid = dbutils.secrets.get(scope="KeyVaultConnection", key="tenantid")
storage_account="tarunadlsproject1"

4 minutes ago (x16) 3
spark.conf.set(
    f"fs.azure.account.auth.type.{storage_account}.dfs.core.windows.net",
    "OAuth"
)

spark.conf.set(
    f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net",
    appid
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.core.windows.net",
    secretid
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.{storage_account}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenantid}/oauth2/token"
)

Just now (x16) 4
Python
adlspath="abfss://bronze@tarunadlsproject1.dfs.core.windows.net/"
```

Notebook 2: Transformation and SCD Type 1 Implementation

Steps:

- We import the notebook 1 in notebook 2 and run it along with notebook 2 for ADLS connectivity
- Then we read the csv files while defining the schemas explicitly.
- Then we perform null check, create addl columns, replace null values, duplicate check and SCD type 1 on any one of the table.
 1. Null check: use filter function
 2. Create new column: use withColumn
 3. Replace null values: use fillna or replace function
 4. Duplicate check: use window function

```
%run /Workspace/Users/tarunvinodh@gmail.com/Notebook-1 ADLS Connection
from pyspark.sql.functions import *
```

```
address_schema = StructType([
    StructField("AddressSK", IntegerType(), True),
    StructField("CustomerID", IntegerType(), True),
    StructField("City", StringType(), True),
])
customers_schema = StructType([
    StructField("CustomerID", IntegerType(), False),
    StructField("FirstName", StringType(), True),
    StructField("Email", StringType(), True)
])
loyalty_schema = StructType([
    StructField("LoyaltyID", IntegerType(), False),
    StructField("CustomerID", IntegerType(), False),
    StructField("PointsBalance", IntegerType(), True),
    StructField("MemberTier", StringType(), True)
])
orders_schema = StructType([
    StructField("OrderID", IntegerType(), False),
    StructField("CustomerID", IntegerType(), False),
    StructField("OrderDate", TimestampType(), True),
    StructField("TotalAmount", DoubleType(), True)
])
tickets_schema = StructType([
    StructField("TicketID", IntegerType(), False),
    StructField("CustomerID", IntegerType(), False),
    StructField("IssueType", StringType(), True),
    StructField("Status", StringType(), True)
])
```

```
df_address=spark.read.format("csv").schema(address_schema).option("header",
"true").load(f"{adlspath}/Addresses/Addresses.csv")
df_customers=spark.read.format("csv").schema(customers_schema).option("header
", "true").load(f"{adlspath}/Customers/Customers")
df_loyalty=spark.read.format("csv").schema(loyalty_schema).option("header","t
rue").load(f"{adlspath}/LoyaltyPoints/LoyaltyPoints")
df_orders=spark.read.format("csv").schema(orders_schema).option("header",
"true").load(f"{adlspath}/Orders/Orders")
```

```
df_tickets=spark.read.format("csv").schema(tickets_schema).option("header",
"true").load(f"{adlspath}/SupportTickets/SupportTickets")
```

Null Check:

```
df_address=df_address.filter(df_address.AddressSK.isNotNull())
df_customers=df_customers.filter(df_customers.CustomerID.isNotNull())
df_loyalty=df_loyalty.filter(df_loyalty.LoyaltyID.isNotNull())
df_orders=df_orders.filter(df_orders.OrderID.isNotNull())
df_tickets=df_tickets.filter(df_tickets.TicketID.isNotNull())
```

Create New Column:

```
df_address=df_address.withColumn("Country", when(col("City") == "New York",
"USA").when(col("City") == "Manchester", "UK").when(col("City") == "Toronto",
"Canada").otherwise("Germany"))
```

```
df_customers=df_customers.withColumn("LastName",when(col("FirstName") ==
"Alice", "Smith").when(col("FirstName") == "Bob",
"Jones").when(col("FirstName") == "Charlie",
"Williams").otherwise("Unknown"))
```

Replace Null Values:

```
df_loyalty=df_loyalty.fillna({
    "MemberTier": "N/A",
    "PointsBalance": -1
})
df_loyalty.display()
```

Duplicate Check:

```
df_customers=df_customers.dropDuplicates(["CustomerID"])
df_tickets=df_tickets.dropDuplicates(["TicketID"])
df_customers.display()
df_tickets.display()
from pyspark.sql import Window
```

```
w = Window.partitionBy("CustomerID").orderBy(
    when((col("PointsBalance") == -1) | (col("MemberTier") == "N/A"),
1).otherwise(0)
)
df_loyalty_with_rn = df_loyalty.withColumn("row_num", row_number().over(w))
df_loyalty=df_loyalty_with_rn.filter(col("row_num") == 1).drop("row_num")
df_loyalty.display()
```

```
w1 = Window.partitionBy("CustomerID").orderBy(col("OrderDate").desc())
df_orders_with_rn = df_orders.withColumn("row_num", row_number().over(w1))
df_orders_with_rn.display()
df_orders=df_orders_with_rn.filter(col("row_num") == 1).drop("row_num")
df_orders.display()
```

Save as Managed Delta Table in DataBricks:

```
df_address.write.format("delta").saveAsTable("address")
df_customers.write.format("delta").saveAsTable("customers")
df_loyalty.write.format("delta").saveAsTable("loyalty")
df_orders.write.format("delta").saveAsTable("orders")
df_tickets.write.format("delta").saveAsTable("tickets")
```

SCD TYPE 1 for ADDRESS Table:

```
%sql
Create Table if not exists address_scdtype1
(
    AddressSK int,
    CustomerID int,
    City string,
    Country String,
    Hashvalue bigint,
    CreatedDate timestamp,
    UpdatedDate timestamp,
    CreatedBy string,
    UpdatedBy string
)
```

```
from delta.tables import DeltaTable
table_name='address_scdtype1'
delta_tgt=DeltaTable.forName(spark, table_name)
```

```
from pyspark.sql.functions import *
df_hash=df_address.withColumn('src_hash',crc32(concat(col('AddressSK'),col('CustomerID'),col('City'),col('Country'))))
df_hash.display()
```

```
(
delta_tgt.alias("tgt").merge(
    df_hash.alias("src"),
    "tgt.CustomerID = src.CustomerID",
).whenMatchedUpdate(condition="tgt.Hashvalue!=src.src_hash",
    set={
        "tgt.AddressSK":"src.AddressSK",
        "tgt.CustomerID":"src.CustomerID",
        "tgt.City":"src.City",
        "tgt.Country":"src.Country",
        "tgt.Hashvalue":"src.src_hash",
        "tgt.UpdatedDate":current_timestamp(),
        "tgt.UpdatedBy":lit("Databricks"),
        "tgt.CreatedDate":current_timestamp(),
        "tgt.CreatedBy":lit("Databricks")
    }).whenNotMatchedInsert(values={
        "tgt.AddressSK":"src.AddressSK",
        "tgt.CustomerID":"src.CustomerID",
        "tgt.City":"src.City",
        "tgt.Country":"src.Country",
        "tgt.Hashvalue":"src.src_hash",
        "tgt.CreatedDate":current_timestamp(),
        "tgt.CreatedBy":lit("Databricks"),
        "tgt.UpdatedDate":lit("9999-12-31"),
        "tgt.UpdatedBy":lit("N/A"),
    }).execute()
)
```


Notebook 3: Revenue and Profit Analysis

Steps:

- We create manual dataframe with some random sales data.
- Then we calculate Revenue, Profit, Highest Cost per Product, Customer who bought Highest number of products

```
from pyspark.sql import *  
from pyspark.sql.functions import *
```

```
data = [  
    (1, 5, 100.0, 70.0, "Laptop", "John"),  
    (2, 3, 200.0, 150.0, "Mobile", "Alice"),  
    (3, 10, 50.0, 30.0, "Headphones", "Bob"),  
    (4, 7, 120.0, 90.0, "Tablet", "John"),  
    (5, 15, 20.0, 10.0, "Mouse", "Alice"),  
    (6, 8, 80.0, 60.0, "Keyboard", "Bob"),  
    (7, 4, 150.0, 100.0, "Monitor", "David"),  
    (8, 20, 15.0, 8.0, "USB Cable", "Alice")  
]  
  
columns = ["sales_id", "sales_quantity", "selling_price", "cost",  
           "product_name", "customer_name"]  
  
df_sales = spark.createDataFrame(data, columns)  
  
df_sales.display()
```

Caluculate Revenue and Profit:

```
df_sales_calc = df_sales .withColumn("revenue", col("sales_quantity") *  
col("selling_price")) .withColumn("profit", (col("selling_price") -  
col("cost")) * col("sales_quantity"))
```

```
df_sales_calc.display()
```

Store as Delta Table:

```
df_sales_calc.write .format("delta") .saveAsTable("sales_delta_table")
```

Highest Cost per Product:

```
highest_cost_per_product = df_sales_calc.groupBy("product_name")  
.agg(max("cost").alias("highest_cost"))  
highest_cost_per_product.display()
```

Customer who bought highest Number of Products:

```
customer_highest_items =  
df_sales_calc.groupBy("customer_name").agg(sum("sales_quantity").alias("total  
_items_purchased")).orderBy(col("total_items_purchased").desc())  
customer_highest_items.display()
```